MODULE–1 | QUESTION 1

Q1. What is Deep Learning? Give the differences between AI, ML and DL. (8 Marks)

1. Definition of Deep Learning

Deep Learning (DL) is a subset of Machine Learning that uses multi-layered artificial neural networks to automatically learn complex patterns and representations from large amounts of data.

2. Relationship between AI, ML and DL

Artificial Intelligence, Machine Learning and Deep Learning have the following relationship:

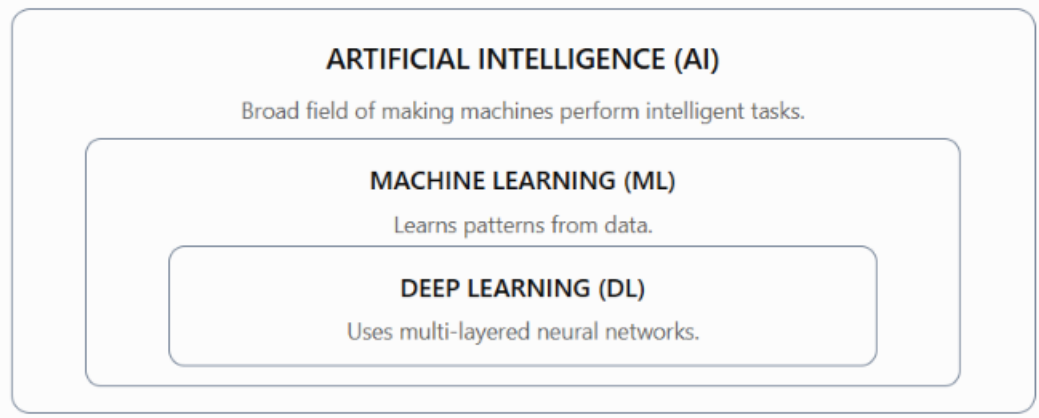


Diagram: AI, ML and DL relationship (nested subsets).

Aspect	AI	ML	DL
Definition	Broad field of intelligent machines.	Subset of AI that learns from data.	Subset of ML using multi-layer neural networks.
Learning Method	Uses rules, reasoning and learning.	Learns patterns using algorithms.	Learns hierarchical patterns using neural networks.
Feature Extraction	Uses rules or other methods.	Usually requires manual feature extraction.	Automatically learns useful features.
Data Requirement	Depends on the method.	Often works with smaller datasets.	Generally benefits from large datasets.
Computational Requirement	Depends on the application.	Generally less computationally expensive.	Usually computationally expensive.

Examples	Expert systems, AI assistants.	SVM, Decision Tree.	CNN, RNN.

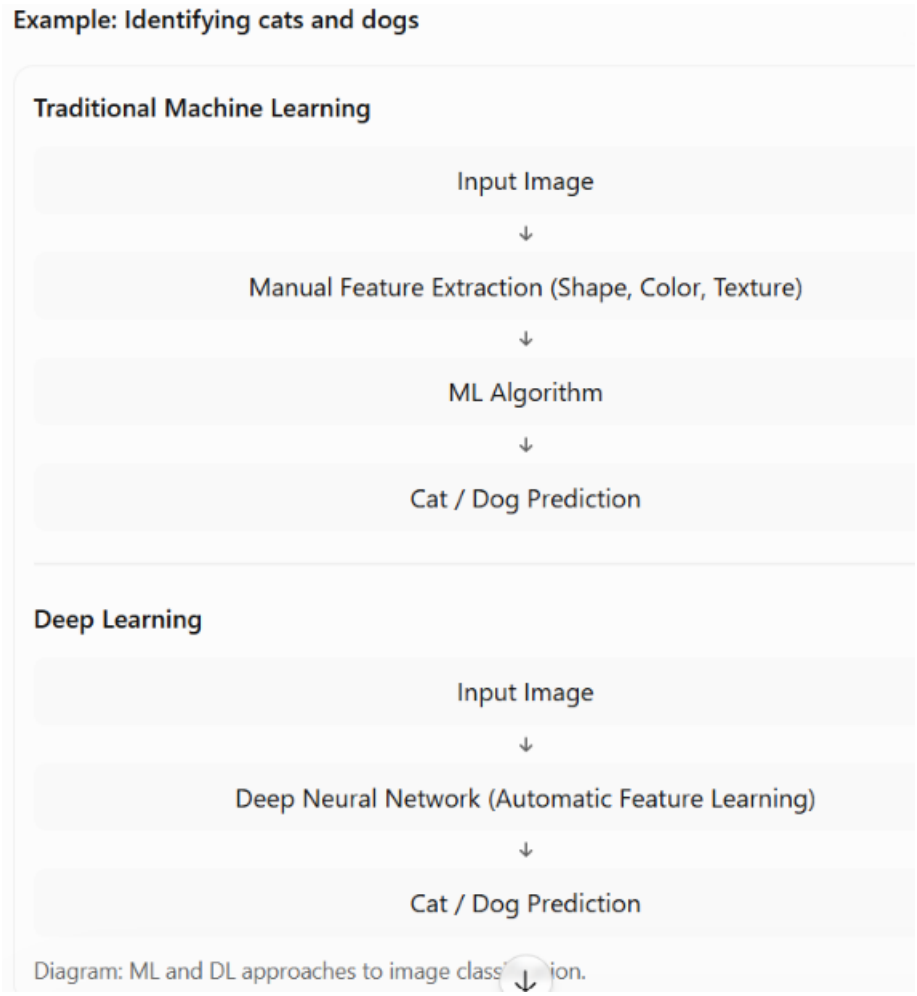


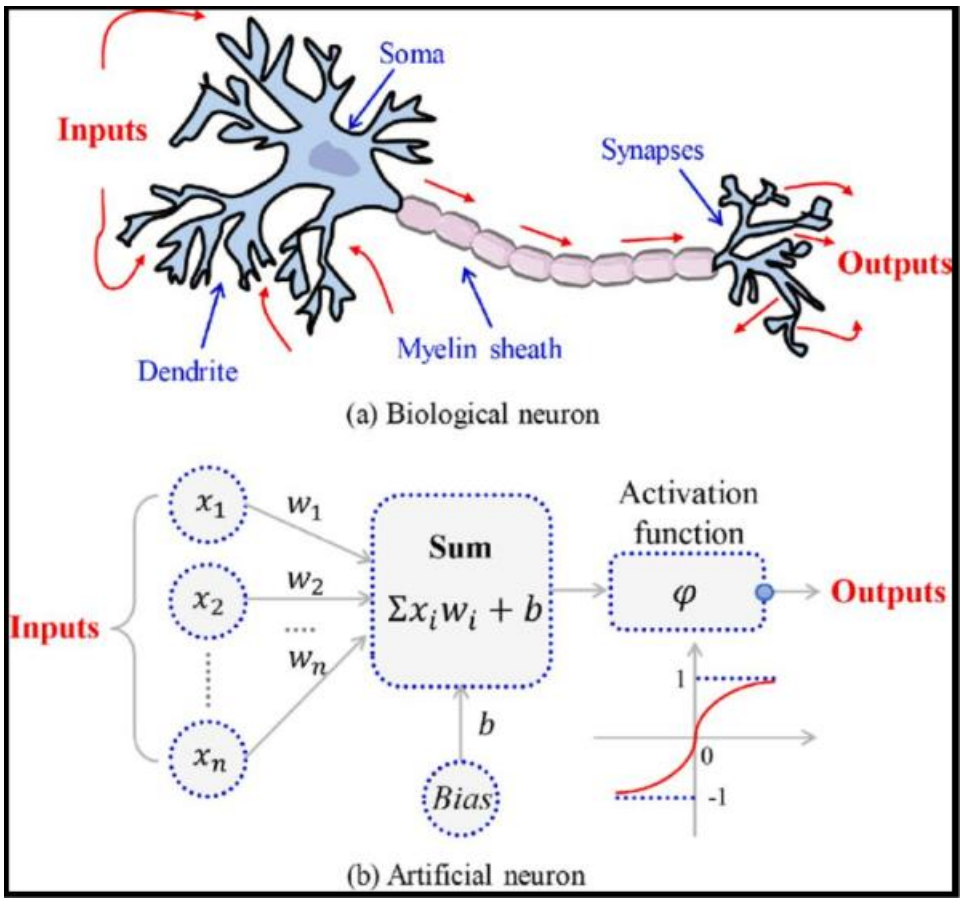
Diagram: ML and DL approaches to image classification.

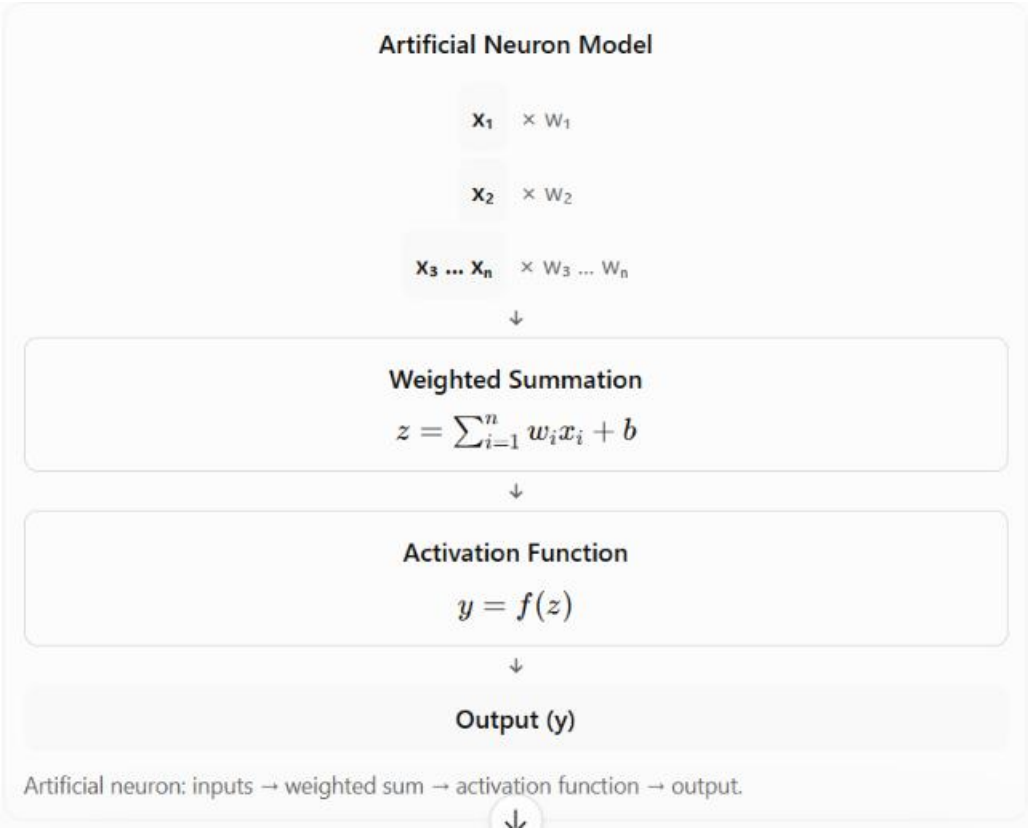
Q2. Explain Artificial Neuron with respect to Biological Neuron in the form of a neat diagram representation. (8 Marks)

1. Introduction

The biological neuron is the basic functional unit of the human nervous system. It receives signals, processes them and transmits the output to other neurons.

An artificial neuron is a mathematical model inspired by the biological neuron. It receives inputs, applies weights, calculates a weighted sum and uses an activation function to produce an output.





3. Components of a Biological Neuron

A biological neuron consists of the following main parts:

1. Dendrites: Receive signals from other neurons.
2. Cell Body (Soma): Processes and combines the received signals.
3. Synapses: Connections through which signals are transmitted; synaptic strength influences signal transmission.
4. Axon: Transmits signals from the neuron to other cells.

4. Components of an Artificial Neuron

An artificial neuron consists of the following components:

1. Inputs (x_1, x_2, ..., x_n): Represent the information received by the neuron.
2. Weights (w_1, w_2, ..., w_n): Determine the importance of each input.
3. Weighted Summation: Calculates the sum of the weighted inputs and bias.
4. Bias (b): A constant added to the weighted sum to shift the decision boundary.
5. Activation Function: Transforms the weighted sum into an output.
6. Output (y): The final result produced by the artificial neuron.

Biological Neuron	**Artificial Neuron**
Dendrites	Inputs
Synaptic strength	Weights
Cell body	Weighted summation
Activation	Activation function
Axon	Output

Q3. Discuss the following:

1) Single Layer Perceptron

2) Learning Rule

1. Single Layer Perceptron

A Single Layer Perceptron (SLP) is the simplest type of artificial neural network consisting of an input layer and an output layer, with no hidden layer. It is used for binary classification of linearly separable data.

The perceptron calculates the weighted sum of inputs and applies an activation function to produce the output.

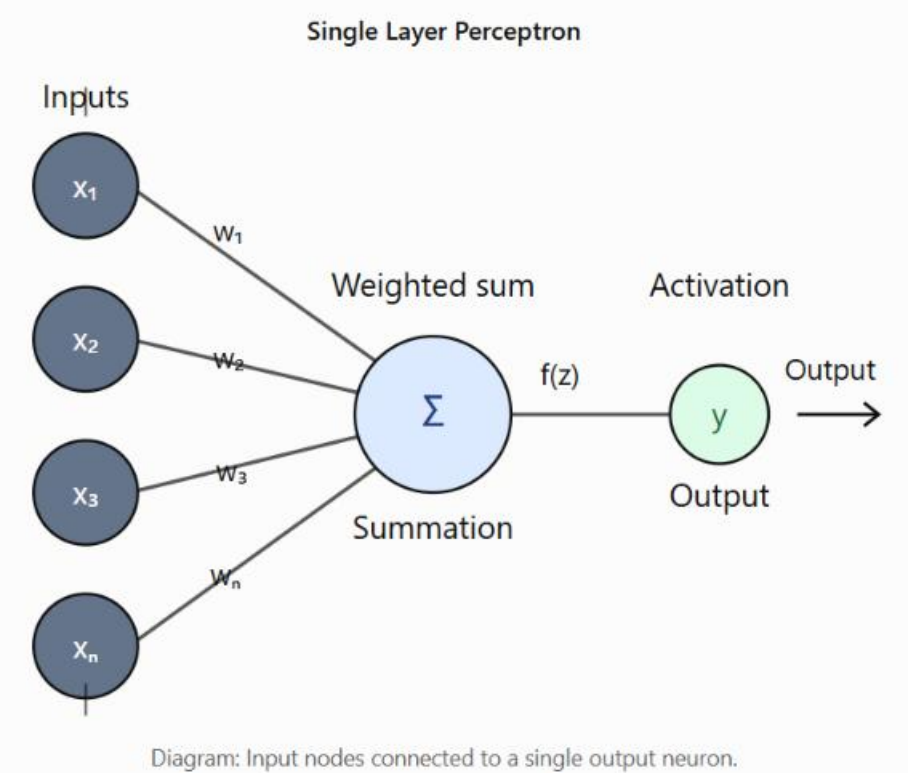


Diagram: Input nodes connected to a single output neuron.

Working of Single Layer Perceptron

1. The input layer receives input values $x_1, x_2, \ldots, x_n$.
2. Each input is multiplied by its corresponding weight.
3. The weighted inputs are added with the bias.
4. The activation function is applied to the weighted sum.
5. The final output is generated.

Mathematical Equation

The weighted sum is:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

The output is:

$$y = f(z)$$

Where:

- x_i= Input
- w_i= Weight
- b= Bias
- $f(z)$= Activation function
- y= Output

2. Perceptron Learning Rule

Definition

The Perceptron Learning Rule is an algorithm used to update the weights and bias of a perceptron based on the difference between the target output and the predicted output.

The weights are adjusted when the perceptron makes an incorrect prediction.

Learning Rule Equation

Weight update:

$$w_i^{new} = w_i + \eta(t - y)x_i$$

Bias update:

$$b^{new} = b + \eta(t - y)$$

Where:

- w_i= Current weight
- η= Learning rate
- t= Target output
- y= Predicted output
- x_i= Input value
- b= Bias

Steps of Perceptron Learning Rule

1. Initialize the weights and bias.
2. Provide the input training example.
3. Calculate the weighted sum.
4. Apply the activation function to obtain the predicted output.

5. Calculate the error:

$$e = t - y$$

6. Update the weights and bias using the learning rule.
7. Repeat the process for all training examples until the desired classification is achieved.

Example: Perceptron Learning Rule

Given:

Parameter	**Value**
x_1	1
x_2	2
w_1	0.2
w_2	0.3
b	0.1
Target t	0
Learning rate η	0.1

Step 1: Calculate weighted sum

$$z = w_1 x_1 + w_2 x_2 + b$$

$$z = (0.2)(1) + (0.3)(2) + 0.1$$

$$z = 0.2 + 0.6 + 0.1 = 0.9$$

Step 2: Calculate predicted output

Since $z \geq 0$,

$$y = 1$$

Step 3: Calculate error

$$e = t - y$$

$$e = 0 - 1 = -1$$

Step 4: Update weights

For w_1:

$$w_1^{new} = 0.2 + 0.1(-1)(1)$$

$$\boxed{w_1^{new} = 0.1}$$

For w_2:

$$w_2^{new} = 0.3 + 0.1(-1)(2)$$

$$\boxed{w_2^{new} = 0.1}$$

Step 5: Update bias

$$b^{new} = b + \eta(t - y)$$

$$b^{new} = 0.1 + 0.1(-1)$$

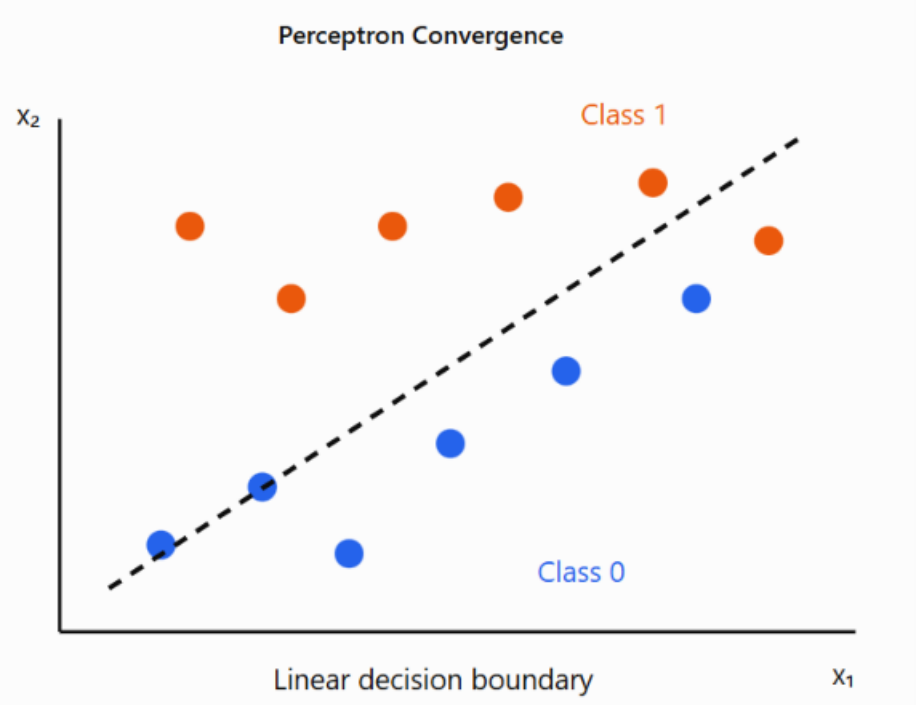
$$\boxed{b^{new} = 0}$$

Q4. Elaborate on Perceptron Convergence Theorem with the limitations of Perceptron. (8 Marks)

Perceptron Convergence Theorem

A perceptron is a single-layer neural network used for binary classification. Its learning algorithm adjusts weights and bias to correctly classify training data.

If the training data is linearly separable, the perceptron learning algorithm will find a set of weights that correctly classifies all training examples in a finite number of updates.



Limitations of Perceptron (Short Answer)

1. Cannot solve non-linear problems: Cannot solve XOR problems because it creates only a linear decision boundary.
2. Limited to linear classification: Cannot learn complex non-linear patterns.
3. No hidden layer: Cannot learn hierarchical features or complex relationships.
4. Binary classification: Basic perceptron is mainly designed for binary classification.
5. May not converge: The algorithm may not converge when data is not linearly separable.

Example: XOR Problem

The XOR truth table is:

x_1	x_2	XOR Output
0	0	0
0	1	1
1	0	1
1	1	0

- Positive class: (0, 1), (1, 0)
- Negative class: (0, 0), (1, 1)

These points cannot be separated by a single straight line. Hence, a single-layer perceptron cannot solve the XOR problem.



Q5. Explain different types of MLP (Feed Forward Neural Networks) with neat diagrams. (8 Marks)

A Multilayer Perceptron (MLP) is a feed-forward artificial neural network consisting of an input layer, one or more hidden layers and an output layer. Each neuron in one layer is generally connected to neurons in the next layer.

Signals flow in the forward direction, from input to output, without feedback connections.

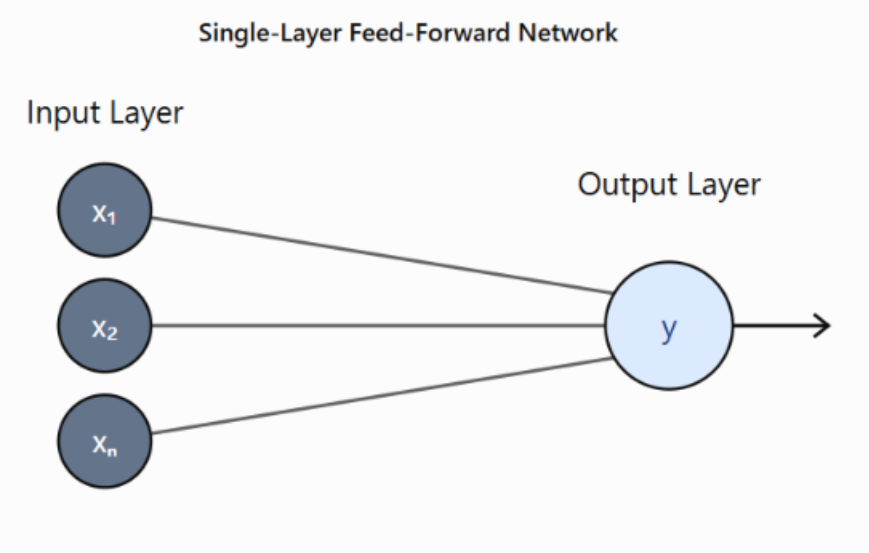
2. Basic Architecture of MLP

An MLP consists of three main components:

1. Input Layer: Receives the input features.
2. Hidden Layer(s): Performs weighted calculations and learns complex patterns.
3. Output Layer: Produces the final prediction.

Type 1: Single-Layer Feed-Forward Network

A single-layer feed-forward network has an input layer directly connected to the output layer. It has no hidden layer.



Working:

- Input features are provided to the network.
- Each input is multiplied by its corresponding weight.
- A bias is added to the weighted sum.
- The result passes through an activation function.
- The activated value produces the final output.

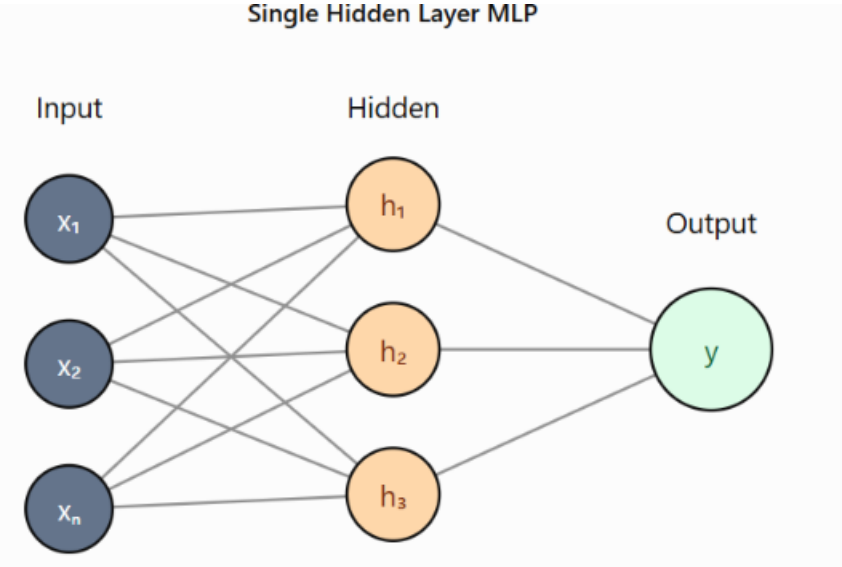
Applications:

- Binary classification.
- Linear regression.
- Simple threshold-based classification.

Type 2: Single Hidden Layer MLP

A single hidden layer MLP consists of an input layer, one hidden layer and an output layer.

The hidden layer enables the network to learn non-linear relationships between input and output.



Characteristics:

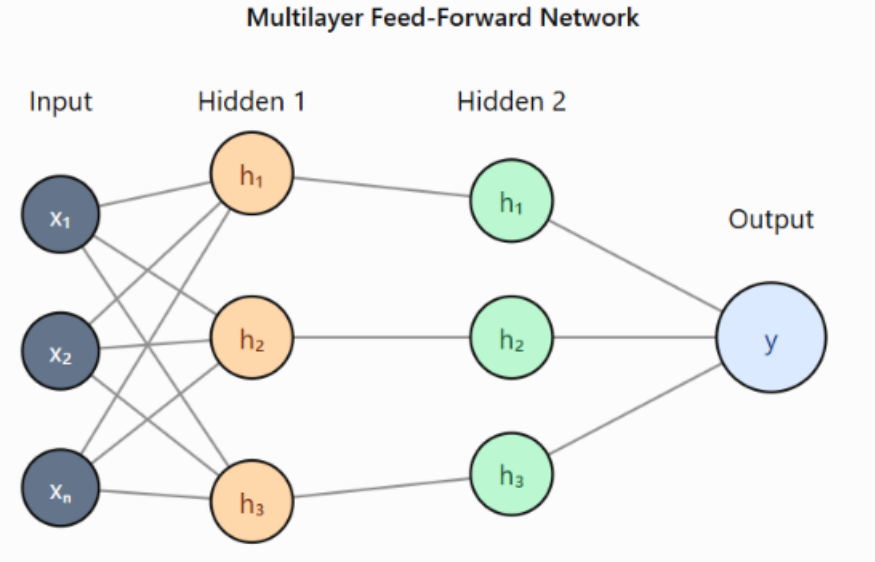
- Contains one hidden layer.
- Uses activation functions in hidden neurons.
- Can learn non-linear decision boundaries.
- Suitable for basic pattern recognition and classification.

Applications:

- Classification.
- Function approximation.
- Pattern recognition.

Type 3: Multilayer / Deep Feed-Forward Network

A multilayer feed-forward network contains two or more hidden layers between the input and output layers. When it has multiple hidden layers, it is commonly called a deep neural network.



Working:

1. Input data enters the input layer.
2. Signals are forwarded to the first hidden layer.
3. Each neuron calculates a weighted sum and applies an activation function.
4. Outputs flow sequentially through the hidden layers.
5. The output layer generates the final prediction.
6. During training, prediction error is propagated backward to update weights.

Applications:

- Image classification.
- Medical diagnosis support.
- Fraud detection.
- Complex pattern recognition.

Q6. Explain Activation Functions with the graphs and relevant equations. (8 Marks)

An activation function is a mathematical function applied to the weighted sum of inputs in a neural network. It determines the output of a neuron and introduces non-linearity, allowing neural networks to learn complex patterns.

The weighted sum is given by:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

The output of the neuron is:

$$y = f(z)$$

Where $f(z)$is the activation function.

2. Need for Activation Functions

1. Introduces non-linearity into neural networks.
2. Helps the network learn complex relationships.
3. Controls the output of neurons.
4. Different activation functions are suitable for different tasks.

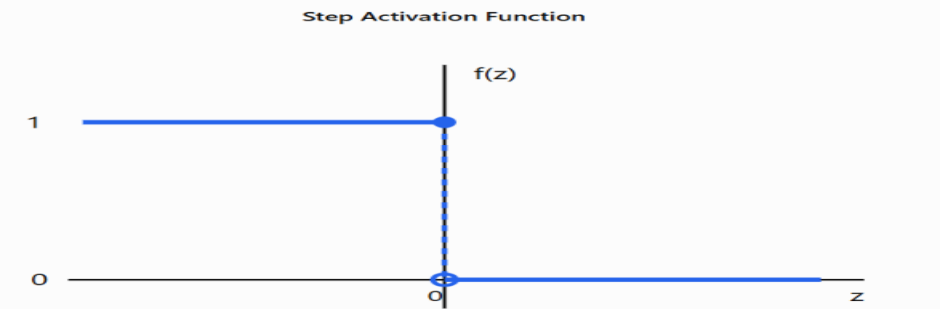
3. Types of Activation Functions

1. Step Function

The step function is commonly used in a basic perceptron for binary classification.

Equation

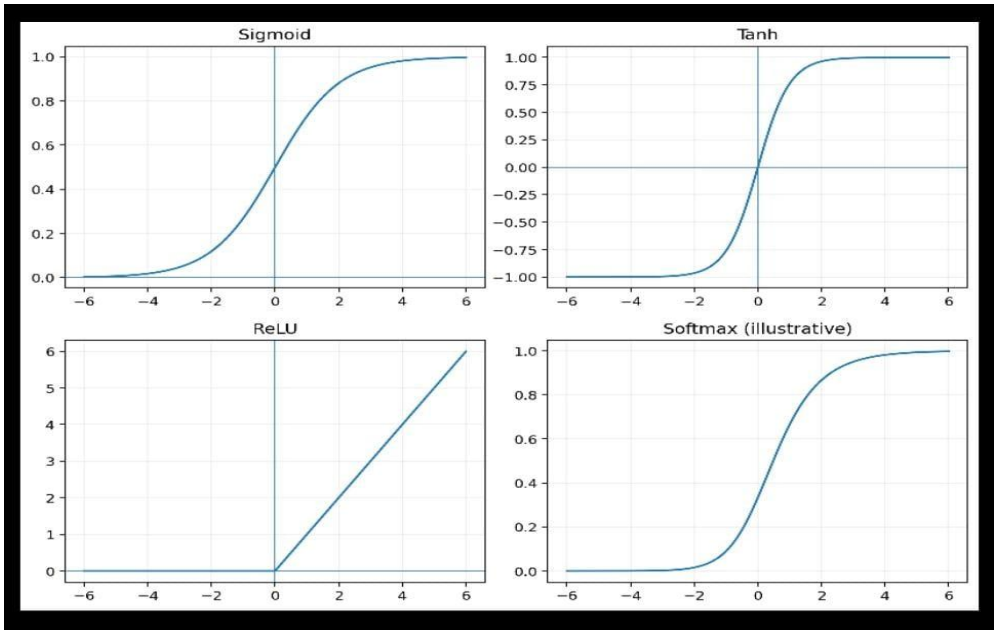
$$f(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$



Graph: Output jumps from 0 to 1 at z = 0.

Advantages:

- Simple to implement.
- Suitable for basic binary classification.



2. Sigmoid Function

The sigmoid function converts the input into a value between 0 and 1. It is often used for binary classification output probabilities.

Equation

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Graph: Smooth S-shaped curve ranging from 0 to 1.

Properties

- Output range: 0to 1.
- Smooth and differentiable.
- Derivative:

$$\sigma'(z) = \sigma(z)[1 - \sigma(z)]$$

Applications: Binary classification and output probabilities.

3. Hyperbolic Tangent (Tanh)

The tanh function maps the input to a value between −1 and +1. It is a zero-centered activation function.

Equation

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Graph: S-shaped curve ranging from −1 to +1.

Properties

- Output range: $(-, 1 + 1)$.
- Zero-centered.
- Smooth and differentiable.
- Derivative:

$$\frac{d}{dz}\tanh(z) = 1 - \tanh^2(z)$$

Applications: Hidden layers in some neural networks, especially when zero-centered activations are useful.

4. ReLU (Rectified Linear Unit)

ReLU is one of the most commonly used activation functions in deep neural networks. It outputs zero for negative inputs and the input itself for positive inputs.

Equation

$$f(z) = \max(0, z)$$

Or,

$$f(z) = \begin{cases} 0 & z < 0 \\ z & z \geq 0 \end{cases}$$

Graph: Zero for negative inputs and linear increase for positive inputs.

Properties

- Output range: $[0, \infty)$.
- Simple and computationally efficient.
- Helps reduce the vanishing-gradient problem for positive inputs.

Applications: Hidden layers in deep neural networks, including many CNNs.

5. Softmax Function

The Softmax function converts a set of scores (logits) into a probability distribution. It is commonly used in the output layer for multi-class classification.

Equation

$$P_i = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$$

Where:

- z_i= Score of class i.
- P_i= Probability of class i.
- n= Number of classes.

Properties

1. Each output probability lies between 0 and 1.
2. The sum of all probabilities is 1.
3. Higher scores receive higher probabilities.

Application: Multi-class classification, such as classifying an image into cat, dog or bird.

Q7. Explain Loss Functions with relevant equations. (8 Marks)

A loss function is a mathematical function that measures the difference between the actual output and the predicted output of a neural network.

It indicates how well the model is performing:

- Low loss: Prediction is close to the actual value.
- High loss: Prediction is far from the actual value.

Need for Loss Functions

1. Measures prediction error.
2. Helps evaluate model performance.
3. Provides feedback during training.
4. Calculates gradients for updating weights and biases.
5. Guides the model toward accurate predictions.

Types of Loss Functions

1. Mean Squared Error (MSE)

Mean Squared Error is commonly used for regression problems. It calculates the average of the squared differences between actual and predicted values.

Equation

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Where:

- y_i= Actual value.
- $\hat{y}_i$= Predicted value.
- n= Number of samples.

Example

Actual values:

$$y = [1, 0, 1]$$

Predicted values:

$$\hat{y} = [0.8, 0.2, 0.6]$$

$$MSE = \frac{(1-0.8)^2 + (0-0.2)^2 + (1-0.6)^2}{3}$$

$$MSE = \frac{0.04 + 0.04 + 0.16}{3}$$

$$\boxed{MSE = 0.08}$$

Advantages:

- Simple to calculate.

Application: Predicting house prices, temperature and sales values.

2. Mean Absolute Error (MAE)

Mean Absolute Error calculates the average absolute difference between actual and predicted values.

Equation

$$MAE = \frac{1}{n}\sum_{i=1}^{n} | y_i - \hat{y}_i |$$

Example

For actual values $[1, 0, 1]$and predicted values $[0.8, 0.2, 0.6]$:

$$MAE = \frac{| 1 - 0.8 | + | 0 - 0.2 | + | 1 - 0.6 |}{3}$$

$$MAE = \frac{0.2 + 0.2 + 0.4}{3}$$

$$\boxed{MAE \approx 0.267}$$

Advantages:

- Less sensitive to outliers than MSE.
- Easy to interpret.

Application: Regression problems.

3. Binary Cross-Entropy Loss

Binary Cross-Entropy is used for binary classification, where the output belongs to one of two classes.

Equation

$$L = -[y\log(\hat{y}) + (1 - y)\log(1 - \hat{y})]$$

For $n$samples:

$$BCE = -\frac{1}{n}\sum_{i=1}^{n}[\, y_i\log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$

Where:

- y= Actual class label, either 0 or 1.
- $\hat{y}$= Predicted probability.

Application: Spam detection, disease detection and binary image classification.

Advantage: Works effectively with sigmoid output probabilities.

4. Categorical Cross-Entropy Loss

Categorical Cross-Entropy is used for multi-class classification when the target is represented using one-hot encoding.

Equation

$$L = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$$

Where:

- C= Number of classes.
- y_i= Actual class value.
- $\hat{y}_i$= Predicted probability for class i.

Example

If the actual class is represented as:

$$y = [1, 0, 0]$$

And the predicted probabilities are:

$$\hat{y} = [0.7, 0.2, 0.1]$$

Then:

$$L = -\log(0.7)$$

$$\boxed{L \approx 0.357}$$

Application: Classifying images into cat, dog or bird.

Advantage: Commonly used with the Softmax activation function.

Q8. Explain Gradient Descent with its algorithm and equation. (8 Marks)

1. Definition

Gradient Descent is an optimization algorithm used in deep learning to minimize the loss function by updating the weights and biases of a neural network.

It finds the direction in which the loss decreases and adjusts the model parameters accordingly.

3. Working Principle

Gradient descent works as follows:

1. Initialize weights and biases.
2. Calculate the model's prediction.
3. Calculate the loss.
4. Find the gradient of the loss.
5. Update the weights in the opposite direction of the gradient.
6. Repeat until the loss is minimized.

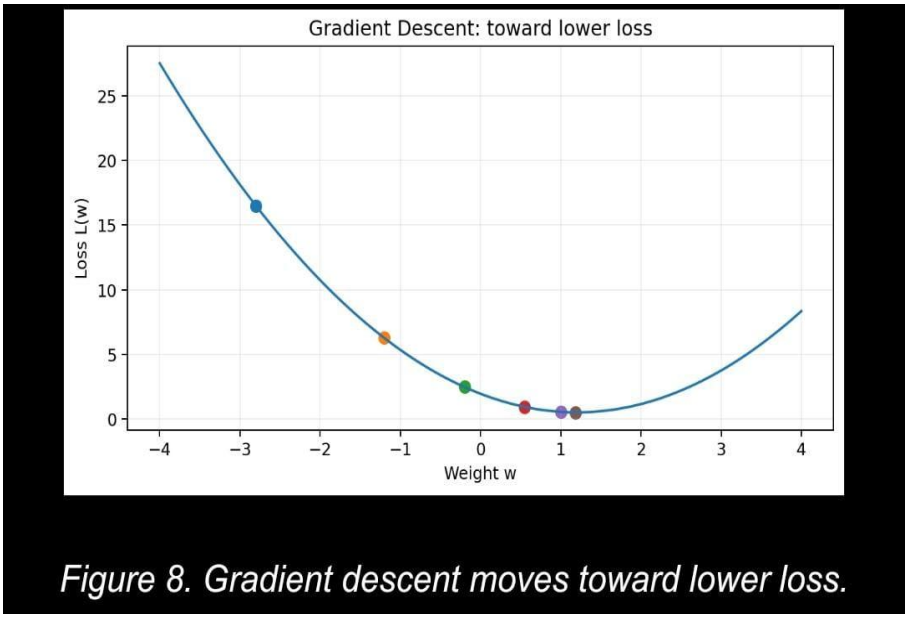


Figure 8. Gradient descent moves toward lower loss.

4. Gradient Descent Equation

The weight update equation is:

$$\boxed{w_{\text{new}} = w - \eta \frac{\partial L}{\partial w}}$$

The bias update equation is:

$$\boxed{b_{\text{new}} = b - \eta \frac{\partial L}{\partial b}}$$

Where:

- w= Current weight.
- η= Learning rate.
- $\frac{\partial L}{\partial w}$= Gradient of loss with respect to weight.
- b= Bias.

Role of Learning Rate

The learning rate ηcontrols the size of each update.

- Small learning rate: Slow learning.
- Large learning rate: May overshoot the minimum.
- Suitable learning rate: Faster and stable convergence.

Types of Gradient Descent

Type	**Description**
Batch Gradient Descent	Uses the entire training dataset for one update
Stochastic Gradient Descent (SGD)	Uses one training example per update
Mini-Batch Gradient Descent	Uses a small batch of training examples

7. Numerical Example

Given:

$$L(w) = (w - 4)^2$$

$$w = 2,\ \eta = 0.1$$

Step 1: Find the gradient

$$\frac{dL}{dw} = 2(w - 4)$$

Substitute $w = 2$:

$$\frac{dL}{dw} = 2(2 - 4)$$

$$\frac{dL}{dw} = -4$$

Step 2: Update the weight

$$w_{\text{new}} = w - \eta \frac{dL}{dw}$$

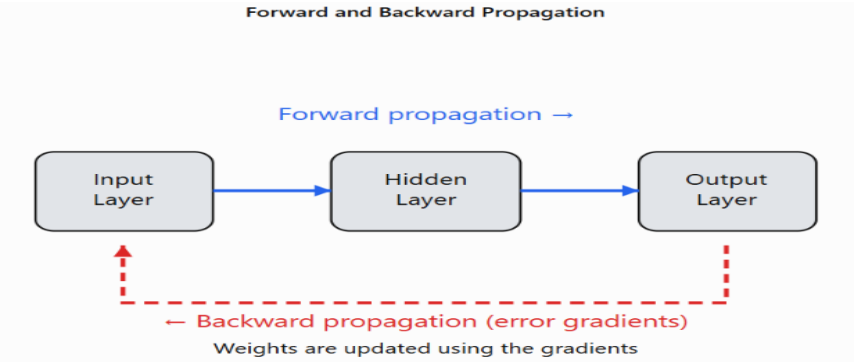
$$w_{\text{new}} = 2 - (0.1)(-4)$$

$$\boxed{w_{\text{new}} = 2.4}$$

Q9. Explain Backpropagation in detail. (8 Marks)

Backpropagation is a supervised learning algorithm used to train artificial neural networks. It calculates the error gradient with respect to each weight by propagating the error backward from the output layer to the input layer.

The calculated gradients are used by Gradient Descent to update the weights and minimize the loss function.



Steps in Backpropagation algorithm(10th question Steps Involved in Training a Neural Network)



Step 1: Initialize Weights

Initialize the weights and biases with small values.

Step 2: Forward Propagation

The input is passed through the network to calculate the predicted output.

For a neuron:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

$$a = f(z)$$

Where $f(z)$is the activation function.

Step 3: Calculate Loss

The difference between the actual output and predicted output is calculated using a loss function.

For Mean Squared Error:

$$L = \frac{1}{2}(y - \hat{y})^2$$

Where:

- y= Actual output.

- $\hat{y}$= Predicted output.

Step 4: Backward Propagation

The error is propagated backward from the output layer to the hidden layers using the chain rule of differentiation.

The gradient is calculated as:

$$\frac{\partial L}{\partial w}$$

Step 5: Update Weights

The weights are updated using gradient descent:

$$\boxed{w_{\text{new}} = w - \eta \frac{\partial L}{\partial w}}$$

The bias is updated as:

$$\boxed{b_{\text{new}} = b - \eta \frac{\partial L}{\partial b}}$$

Step 6: Repeat

The forward and backward propagation process is repeated for several epochs until the loss is minimized.

Chain Rule in Backpropagation

The chain rule is used to calculate the derivative of the loss with respect to a weight.

For a simple neuron:

$$z = wx + b$$

$$a = f(z)$$

The gradient is:

$$\boxed{\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \times \frac{\partial a}{\partial z} \times \frac{\partial z}{\partial w}}$$

Since:

$$\frac{\partial z}{\partial w} = x$$

Therefore:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \times f'(z) \times x$$

This gradient is used to update the weight.

6. Numerical Example

Given:

$$x = 2,\ w = 0.5,\ b = 0$$

Assume a linear activation:

$$a = wx + b$$

Actual output:

$$y = 3$$

Learning rate:

$$\eta = 0.1$$

Step 1: Calculate Prediction

$$a = (0.5)(2) + 0 = 1$$

Step 2: Calculate Loss

$$L = \frac{1}{2}(y - a)^2$$

$$L = \frac{1}{2}(3 - 1)^2 = 2$$

Step 3: Calculate Gradient

$$\frac{\partial L}{\partial w} = (a - y)x$$

$$\frac{\partial L}{\partial w} = (1 - 3)(2)$$

$$\frac{\partial L}{\partial w} = -4$$

Step 4: Update Weight

$$w_{\text{new}} = w - \eta \frac{\partial L}{\partial w}$$

$$w_{\text{new}} = 0.5 - (0.1)(-4)$$

$$\boxed{w_{\text{new}} = 0.9}$$

The weight increases because the prediction is smaller than the actual output.

Advantages

1. Efficiently calculates gradients in multilayer networks.
2. Supports training of deep neural networks.
3. Works with different activation and loss functions.

Limitations

1. Can suffer from vanishing or exploding gradients.
2. Requires differentiable operations in standard implementations.

Q11. Explain the role of Deep Learning in Computer Vision, NLP and Healthcare/Medical Imaging. (8 Marks)

Deep Learning (DL) is a subset of Machine Learning that uses multi-layered artificial neural networks to automatically learn complex patterns from large amounts of data.

Role of Deep Learning in Computer Vision (CV)

Computer Vision is a field of Artificial Intelligence that enables computers to understand and interpret images and videos.

Deep Learning models, especially Convolutional Neural Networks (CNNs), automatically learn features such as edges, shapes, textures and objects from images.

Applications

1. Image Classification: Identifies objects or categories in an image, such as cats and dogs.
2. Object Detection: Detects and locates objects in images or videos.
3. Face Recognition: Identifies or verifies faces.
4. Medical Image Analysis: Detects abnormalities in X-rays, CT scans and MRI images.
5. Self-Driving Vehicles: Identifies roads, vehicles, pedestrians and traffic signs.

Example

Image → CNN → Feature Extraction → Prediction

Role of Deep Learning in Natural Language Processing (NLP)

Natural Language Processing is a branch of AI that enables computers to understand, process and generate human language.

Deep Learning models learn patterns in text, sentences and speech using neural networks such as RNNs, LSTMs and Transformer-based models.

Applications

1. Machine Translation: Converts text from one language to another.
2. Chatbots: Understands user questions and provides responses.
3. Sentiment Analysis: Identifies whether text expresses positive, negative or neutral sentiment.
4. Speech Recognition: Converts spoken language into text.
5. Text Summarization: Generates short summaries of long documents.
6. Question Answering: Provides answers based on given text or questions.

Example

Text/Speech → NLP Model → Language Understanding → Output

Role of Deep Learning in Healthcare and Medical Imaging

Deep Learning is used in healthcare to analyze medical data and assist healthcare professionals in diagnosis, monitoring and treatment planning.

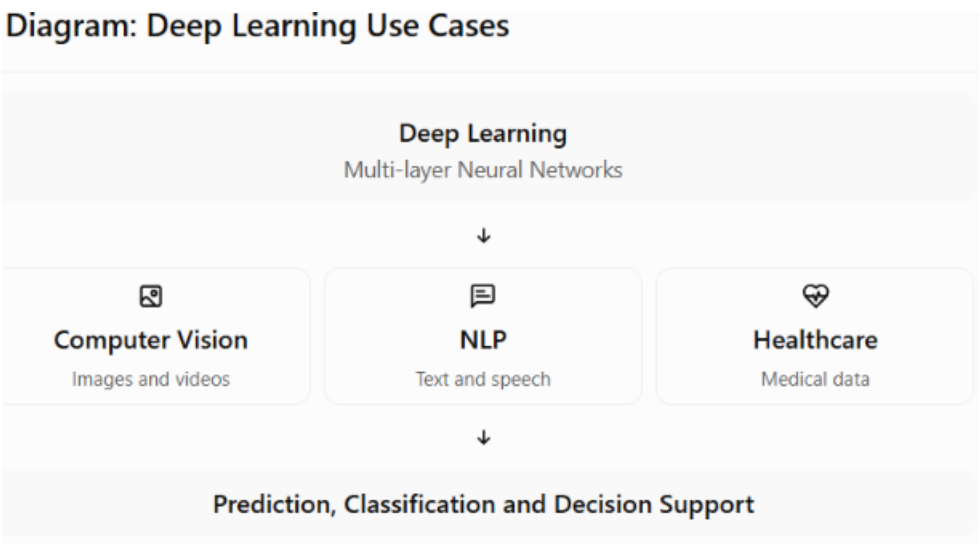
Medical imaging includes X-rays, CT scans, MRI scans and ultrasound images.

Applications

1. Disease Detection: Identifies possible diseases from medical images.
2. Tumor Detection: Helps detect and locate tumors in scans.
3. Medical Image Classification: Classifies medical images into different categories.
4. Image Segmentation: Identifies specific regions, such as organs or abnormal tissues.
5. Patient Monitoring: Helps analyze patient data and detect possible risks.
6. Drug Discovery: Assists in identifying potential drug candidates through pattern analysis.

Example

Medical Image → Deep Learning Model → Analysis → Prediction



MODULE 2 | DEEP LEARNING (BCS702)

Q1. Explain Vanishing Gradients and Exploding Gradients. (8 Marks)

During the training of a deep neural network, backpropagation calculates gradients to update weights. When these gradients become extremely small or extremely large, two problems can occur:

1. Vanishing Gradient
2. Exploding Gradient

2. Vanishing Gradient

The vanishing gradient problem occurs when gradients become extremely small during backpropagation. As a result, the early layers of a deep neural network learn very slowly or stop learning.

Causes

1. Repeated multiplication of small derivatives.
2. Large network depth.
3. Poor weight initialization.

Mathematical Explanation

During backpropagation, the gradient is calculated using the chain rule:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a_n} \cdot \frac{\partial a_n}{\partial a_{n-1}} \cdots \frac{\partial a_1}{\partial w}$$

If each derivative is smaller than 1, their repeated multiplication produces a very small value.

Example

Suppose every derivative is 0.5.

For 10 layers:

$$0.5^{10} = 0.0009765625$$

For 20 layers:

$$0.5^{20} \approx 0.0000009537$$

Effects

- Early layers learn very slowly.
- Weight updates become extremely small.

3. Exploding Gradient

The exploding gradient problem occurs when gradients become extremely large during backpropagation. This leads to unstable weight updates and may cause the loss to diverge.

Causes

1. Large weight values.
2. Poor weight initialization.
3. Very deep or recurrent networks.

Mathematical Explanation

If each derivative is greater than 1, repeated multiplication causes the gradient to increase rapidly:

$$\prod_{i=1}^{n} \frac{\partial a_i}{\partial a_{i-1}} \to \infty$$

Example

Suppose the gradient is multiplied by 3 at every layer.

For a four-layer network:

$$G_1 = 1 \times 3 = 3$$

$$G_2 = 3 \times 3 = 9$$

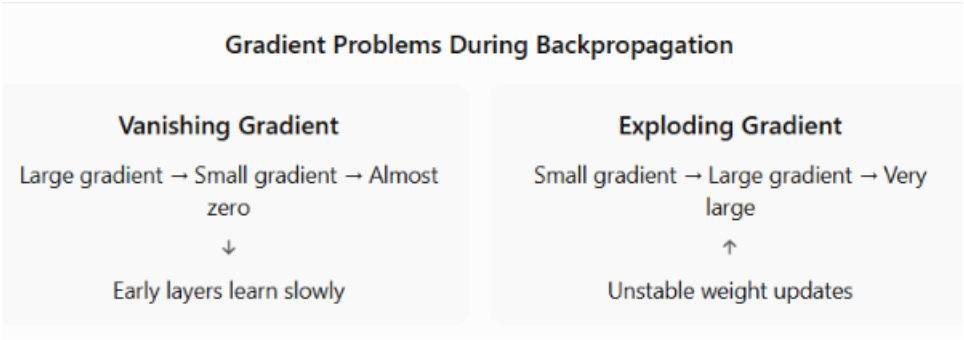
$$G_3 = 9 \times 3 = 27$$

Thus, the gradient reaching the first layer is:

$$\boxed{G_1 = 27}$$

Effects

- Very large weight updates.
- Unstable training.
- Training may diverge.



Q2. Show the importance of L1 and L2 Regularization with examples. (8 Marks)

Regularization is a technique used in Deep Learning to reduce overfitting and improve the model's performance on unseen data.

It adds a penalty to the loss function to prevent the model from learning excessively large or complex weights.

The two important types are:

1. L1 Regularization (Lasso)
2. L2 Regularization (Ridge)

2. L1 Regularization (Lasso)

L1 regularization adds a penalty equal to the sum of the absolute values of the model weights.

Formula

$$L_{\text{total}} = L_{\text{original}} + \lambda \sum_{i=1}^{n} | w_i |$$

Where:

- L_{original}= Original loss.
- λ= Regularization parameter.
- w_i= Model weight.
- $\sum | w_i |$= L1 penalty.

Importance of L1 Regularization

1. Reduces overfitting.

2. Encourages some weights to become exactly zero.
3. Helps in feature selection.

L1 Numerical Example

Given

$$w_1 = 2,\ w_2 = -3,\ w_3 = 4$$

$$\lambda = 0.5$$

Step 1: Calculate L1 penalty

$$L1 = | w_1 | + | w_2 | + | w_3 |$$

$$= | 2 | + | -3 | + | 4 |$$

$$= 2 + 3 + 4 = 9$$

Step 2: Multiply by regularization parameter

$$\lambda L1 = 0.5 \times 9$$

$$\boxed{\text{L1 penalty} = 4.5}$$

L2 Regularization (Ridge)

L2 regularization adds a penalty equal to the sum of the squares of the model weights.

Formula

$$\boxed{L_{\text{total}} = L_{\text{original}} + \lambda \sum_{i=1}^{n} w_i^2}$$

Importance of L2 Regularization

1. Reduces overfitting.
2. Prevents weights from becoming excessively large.
3. Encourages smooth and stable models.

L2 Numerical Example

Given

$$w_1 = 2,\ w_2 = -3,\ w_3 = 4$$

$$\lambda = 0.5$$

Step 1: Calculate L2 penalty

$$L2 = w_1^2 + w_2^2 + w_3^2$$

$$= 2^2 + (-3)^2 + 4^2$$

$$= 4 + 9 + 16 = 29$$

Step 2: Multiply by regularization parameter

$$\lambda L2 = 0.5 \times 29$$

$$\boxed{\text{L2 penalty} = 14.5}$$

Q3. What are Dropout and Early Stopping? Explain their importance. (8 Marks)

Dropout and Early Stopping are regularization techniques used in Deep Learning to reduce overfitting.

Overfitting occurs when a model performs well on training data but performs poorly on unseen data.

- Dropout: Randomly deactivates neurons during training.
- Early Stopping: Stops training when validation performance stops improving.

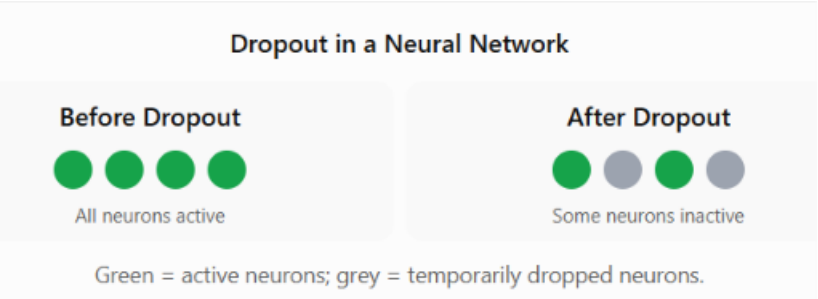
Dropout

Dropout is a regularization technique in which randomly selected neurons are temporarily deactivated during each training iteration.

The dropped neurons do not participate in forward propagation or backpropagation for that iteration.

Working of Dropout

1. Select a dropout rate, such as 0.2 or 0.5.
2. Randomly deactivate neurons during training.
3. Perform forward propagation using the remaining neurons.
4. Calculate the loss and perform backpropagation.
5. Activate a different random set of neurons in the next iteration.
6. During testing, use the complete network with appropriate scaling.



Dropout Formula

If the dropout probability is p, the probability of retaining a neuron is:

$$q = 1 - p$$

With inverted dropout, an active neuron is scaled by:

$$\frac{1}{q}$$

This maintains the expected activation magnitude during training.

Example

If a layer contains 10 neurons and the dropout rate is 0.2:

$$10 \times 0.2 = 2$$

Approximately 2 neurons are dropped in a particular training pass, while the remaining neurons continue processing the input.

Importance of Dropout

1. Reduces overfitting.
2. Prevents excessive dependence on particular neurons.
3. Encourages the network to learn more robust features.

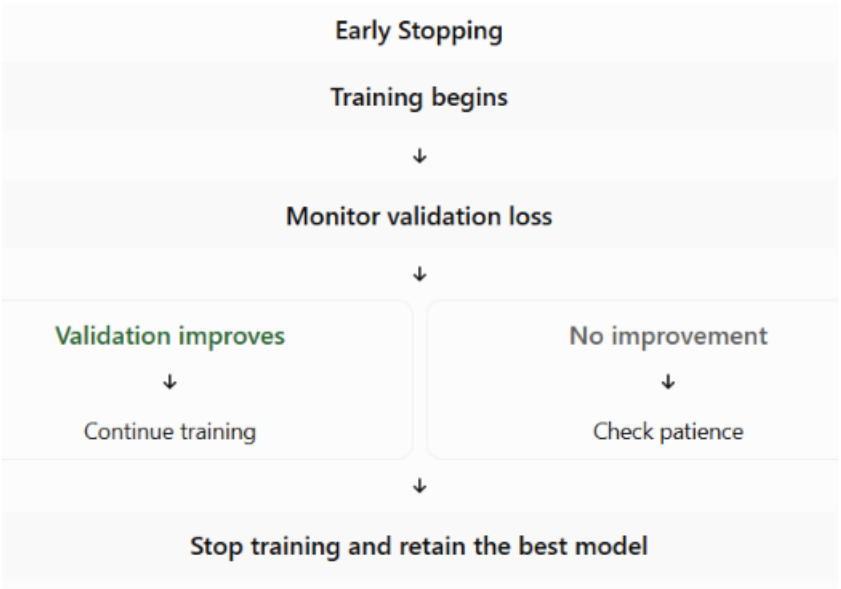
Early Stopping

Early Stopping is a regularization technique that stops model training when validation performance no longer improves.

It prevents the model from continuing to learn training-specific patterns after its generalization performance begins to deteriorate.

Working of Early Stopping

1. Divide the dataset into training and validation sets.
2. Train the neural network for one or more epochs.
3. Measure the validation loss after each epoch.
4. Monitor whether validation loss improves.
5. Stop training if validation loss does not improve for a specified number of epochs.
6. Restore the model parameters from the best validation result, if configured.



Important Terms

- Validation loss: Error calculated using validation data.
- Patience: Number of epochs to wait for improvement.
- Best model: Model parameters corresponding to the best validation performance.

Q4. What is Batch Normalization? Explain its working and importance. (8 Marks)

Batch Normalization normalizes the outputs of a neural network layer using the mean and variance of a mini-batch.

It then applies two learnable parameters:

- γ: Scale parameter.
- β: Shift parameter.

Working of Batch Normalization

Step 1: Calculate the Mean

For a mini-batch:

$$X = [2, 4, 6, 8]$$

The mean is:

$$\mu_B = \frac{2 + 4 + 6 + 8}{4}$$

$$\boxed{\mu_B = 5}$$

Step 2: Calculate the Variance

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4}$$

$$\sigma_B^2 = \frac{9 + 1 + 1 + 9}{4}$$

$$\boxed{\sigma_B^2 = 5}$$

Step 3: Normalize the Values

The normalization formula is:

$$\boxed{\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}}$$

Where $\epsilon$is a small value added for numerical stability.

Step 4: Scale and Shift

The normalized value is transformed using:

$$\boxed{y_i = \gamma \hat{x}_i + \beta}$$

Here, $\gamma$and $\beta$are learned during training.

4. Numerical Example

Given:

$$X = [2, 4, 6, 8],\ \mu_B = 5,\ \sigma_B^2 = 5$$

Ignoring the very small $\epsilon$for simplicity:

Input x	**Normalized value**
2	$-3/\sqrt{5} \approx -1.34$
4	$-1/\sqrt{5} \approx -0.45$
6	$1/\sqrt{5} \approx 0.45$
8	$3/\sqrt{5} \approx 1.34$

The values are centered around zero and have a more controlled scale.

Importance and Advantages of Batch Normalization (Short)

1. Faster Training: Helps the network train faster.
2. Training Stability: Keeps activations at a controlled scale.
3. Higher Learning Rate: Allows larger learning rates in suitable conditions.
4. Less Initialization Sensitivity: Reduces dependence on initial weights.
5. Reduces Overfitting: Batch statistics introduce noise that may provide a regularizing effect.

Q5. Explain Various Weight Initialization Methods

Weight initialization is the process of assigning initial values to the weights of a neural network before training begins.

Weight Update Equation

$$W_{\text{new}} = W_{\text{old}} - \eta \frac{\partial L}{\partial W}$$

2. Zero Initialization

In this method, all weights are initialized to zero.

$$W = 0$$

Problem: All neurons receive identical gradients and learn the same features. This is called the symmetry problem.

Applications:

- Suitable for initializing biases.
- Not recommended for weights in hidden layers.

3. Random Initialization

Weights are initialized using small random values.

$$W \sim N(0, \sigma^2)$$

Example:

$$W = [0.01, -, 0.030.02]$$

Advantages:

- Breaks symmetry between neurons.
- Allows neurons to learn different features.

4. Xavier or Glorot Initialization

Xavier initialization maintains approximately equal activation variance across layers.

Normal Distribution

$$W \sim N\left(0, \frac{2}{n_{in} + n_{out}}\right)$$

Uniform Distribution

$$W \sim U\left(-, \sqrt{\frac{6}{n_{in} + n_{out}}} \sqrt{\frac{6}{n_{in} + n_{out}}}\right)$$

Where:

- n_{in}= Number of input neurons

- n_{out}= Number of output neurons

Best suited for:

- Sigmoid activation
- Tanh activation
- Softmax or linear layers

Example

If $n_{in} = 4$and $n_{out} = 2$:

$$Var(W) = \frac{2}{4+2} = \frac{1}{3}$$

5. He or Kaiming Initialization

He initialization is mainly designed for:

- ReLU
- Leaky ReLU
- ReLU-like activation functions

He Normal

$$W \sim N\left(0, \frac{2}{n_{in}}\right)$$

Standard deviation:

$$\sigma = \sqrt{\frac{2}{n_{in}}}$$

He Uniform

$$W \sim U\left(-, \sqrt{\frac{6}{n_{in}}}\sqrt{\frac{6}{n_{in}}}\right)$$

Example

If $n_{in} = 100$:

$$\sigma^2 = \frac{2}{100} = 0.02$$

$$\sigma = \sqrt{0.02} \approx 0.1414$$

6. Orthogonal Initialization

In orthogonal initialization, the weight matrix satisfies:

$$W^T W = I$$

Where $I$is the identity matrix.

Example

$$W = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Applications:

- Recurrent Neural Networks (RNNs)
- Deep linear networks

7. Sparse Initialization

In sparse initialization, most weights are initialized to zero, while a small number of weights receive random non-zero values.

Example

$$W = \begin{bmatrix} 0 & 0.1 & 0 \\ 0 & 0 & -0.2 \\ 0.05 & 0 & 0 \end{bmatrix}$$

Benefits:

- Reduces the number of initially active connections.

Q6. Outline Various Optimization Techniques

Optimization is the process of adjusting neural network weights to minimize the loss function and improve model performance.

The optimizer updates weights using gradients calculated during backpropagation.

2. Batch Gradient Descent (BGD)

Batch Gradient Descent calculates the gradient using the entire training dataset before updating the weights.

Working

1. Take all training samples.
2. Calculate the loss.
3. Compute the gradient.
4. Update the weights.

Advantages

- Stable and accurate gradient.

Disadvantages

- Requires high memory.

3. Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent updates the weights using one training sample at a time.

Working

1. Select one training sample.
2. Calculate the loss and gradient.
3. Update the weights immediately.
4. Repeat for all samples.

Advantages

- Faster updates.
- Requires less memory.

Disadvantages

- Learning is noisy.

4. Momentum Gradient Descent

Momentum uses the previous update direction to accelerate learning and reduce oscillations.

Formula

$$v_t = \beta v_{t-1} + (1 - \beta)\nabla L$$

$$W_t = W_{t-1} - \eta v_t$$

Where:

- v_t= Velocity
- β= Momentum coefficient
- η= Learning rate

Advantages

- Speeds up convergence.
- Reduces oscillations.

5. Nesterov Accelerated Gradient (NAG)

Nesterov Accelerated Gradient calculates the gradient after looking ahead in the direction of momentum.

Working

1. Estimate the future position using momentum.
2. Calculate the gradient at that future position.
3. Update the weights.

Advantage

It provides better anticipation of the gradient direction and can improve convergence compared with ordinary momentum.

6. AdaGrad

AdaGrad assigns an individual learning rate to each parameter.

- Frequently updated parameters receive smaller learning rates.
- Infrequently updated parameters receive relatively larger learning rates.

Formula

$$G_t = G_{t-1} + g_t^2$$

$$W_{t+1} = W_t - \frac{\eta}{\sqrt{G_t + \epsilon}} g_t$$

Advantages

- Uses adaptive learning rates.
- Useful for sparse data and features.

7. RMSProp

RMSProp improves AdaGrad by using an exponentially moving average of squared gradients.

Formula

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1-\beta) g_t^2$$

$$W_{t+1} = W_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t$$

Advantages

- Works well for non-stationary objectives.
- Reduces oscillations.

8. Adam Optimizer

Adam (Adaptive Moment Estimation) combines the benefits of Momentum and RMSProp.

It maintains:

1. First moment – average of gradients.
2. Second moment – average of squared gradients.

Formulas

First moment:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

Second moment:

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

Bias correction is applied:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

Weight update:

$$W_{t+1} = W_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Advantages

- Adaptive learning rate.
- Commonly used in deep learning.

MODULE 3: Convolutional Neural Networks (CNN)

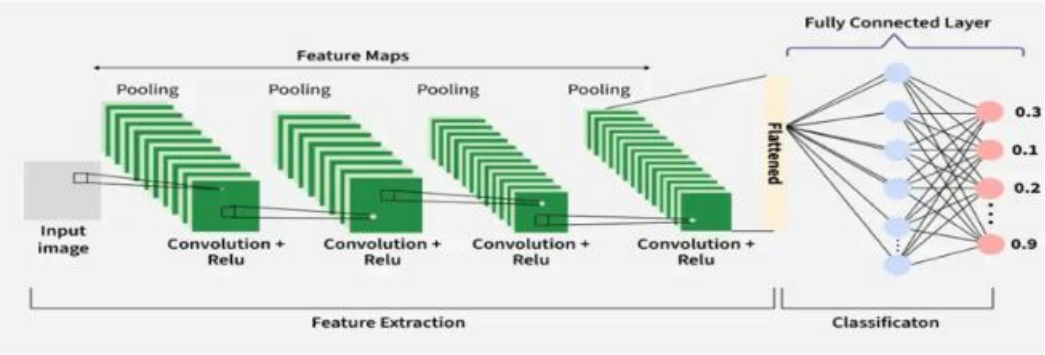
Q1. What is CNN? Explain the Motivation of CNN in Image Processing

1. Definition of CNN

A Convolutional Neural Network (CNN) is a deep learning architecture mainly used for processing images and other spatial data.

CNN automatically extracts important features such as edges, textures, shapes, and objects using convolution operations. It captures spatial relationships between pixels.

Architecture of CNN



Motivation of CNN in Image Processing (Easy Way)

1. Automatic Feature Extraction: CNN automatically learns features from images.

- First layers detect edges and lines.
- Middle layers detect shapes and textures.
- Deep layers detect objects.

2. Preserves Spatial Information: CNN understands the position and relationship of pixels in an image.

3. Local Connectivity: CNN processes small parts of an image instead of the entire image at once, reducing computation.

4. Parameter Sharing: The same filter is used across the image.

- Reduces the number of weights.
- Detects the same feature in different locations.

5. Translation Handling: CNN can recognize an object even when its position changes in the image.

6. Hierarchical Learning: CNN learns features step by step, from simple edges to complex objects.

7. Dimensionality Reduction: Pooling reduces the size of feature maps, lowering computation while retaining important features.

Q2. Explain Padding, Stride and Pooling in CNN

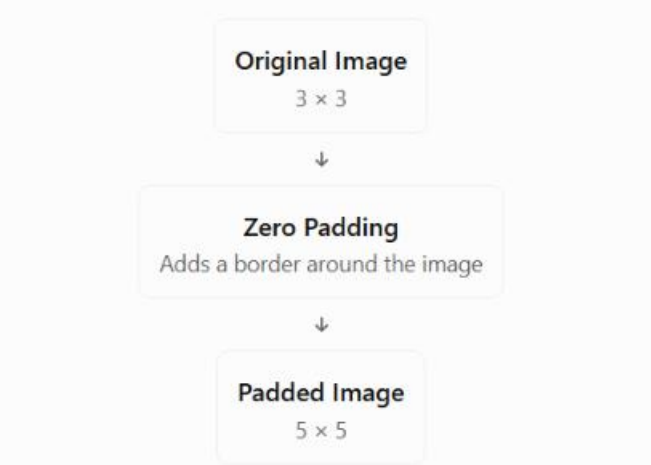
Padding, stride, and pooling are important operations in CNN architecture. They control the size of feature maps, reduce computation, and help extract important image features.

A. Padding

Padding is the process of adding extra pixels, usually zeros, around the boundary of an input image before convolution.

Types of Padding

1. Valid Padding: No padding is added.
2. Same Padding: Padding is added to maintain the spatial dimensions, generally when stride is 1.



Importance of Padding

- Preserves image boundary information.
- Controls the output feature-map size.
- Helps maintain spatial dimensions.

B. Stride

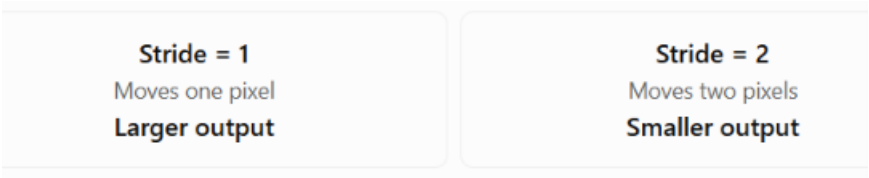
Stride is the number of pixels by which a convolution filter moves across the input image during each step.

Example

- Stride = 1: Filter moves one pixel at a time.
- Stride = 2: Filter moves two pixels at a time.

A larger stride produces a smaller output feature map.

Diagram



Output Size Formula

$$Output = \left\lfloor \frac{N + 2P - F}{S} \right\rfloor + 1$$

Where:

- N= Input size
- P= Padding
- F= Filter size
- S= Stride

Example

Given:

- Input = 5×5
- Filter = 3×3
- Padding = 0
- Stride = 1

$$Output = \frac{5-3}{1} + 1 = 3$$

Therefore, output size is:

$$\boxed{3 \times 3}$$

C. Pooling

Pooling is a downsampling operation used to reduce the spatial dimensions of feature maps while retaining important information.

Types of Pooling

1. Max Pooling

Max pooling selects the maximum value from a region.

Example

Input:

$$\begin{bmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 1 & 2 \\ 7 & 2 & 8 & 3 \\ 4 & 1 & 9 & 5 \end{bmatrix}$$

Using a 2×2filter and stride 2:

$$Output = \begin{bmatrix} 6 & 4 \\ 7 & 9 \end{bmatrix}$$

Advantages:

- Retains the strongest features.
- Reduces spatial dimensions.

2. Average Pooling

Average pooling calculates the average of all values in a region.

For the first 2×2region:

$$Average = \frac{1+3+5+6}{4} = 3.75$$

The average operation is repeated for all regions.